### ICS 202 – Data Structures and Algorithms

# Topic 07: Trees, Binary Trees and Binary Search Trees

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

*Tree notation used below: a binary search tree is given by the order in which its keys were inserted into an initially empty BST; other trees are given by listing each node's children ("X: L, R" means X has left child L and right child R; "X: -, R" means X has only a right child).*

## Part A: Multiple choice

### Q1.

Midterm 222 (Trees compilation)

Given the following two illustrations (edges listed as undirected lines between nodes, drawn top to bottom):

Illustration 1: nodes 5, 3, 2, 4, 1, 6; edges 5–3, 5–2, 3–4, and a separate edge 1–6 (node 1 is not connected to node 5).

Illustration 2: nodes 5, 3, 2, 4, 1, 6; edges 5–3, 5–2, 3–4, 2–4, 2–1, 2–6.

(A) Illustration 1 is a binary tree and illustration 2 is a tree.

(B) Both illustrations are trees.

(C) Both illustrations are not trees.

(D) Illustration 1 is not a binary tree and illustration 2 is a tree.

**Answer:** (C). Illustration 1 has no unique path from the root to 1 and 6 (two parts); in illustration 2 node 4 has two parents (3 and 2).

My answer

### Q2.

Midterm 222 (Trees compilation)

Given the following tree: A: B, C; B: D; D: H; C: E, F, G (three children); E: I, J.

Choose the true statements from the following: i. Node C has a degree of three. ii. Node D is the parent of Node H. iii. E and G are siblings. iv. Nodes F, G, H, I, and J are leaves.

(A) All the statements are true.

(B) None of the statements is true.

(C) Only statements i and ii are true.

(D) Only statements iii and iv are true.

**Answer:** (A). C has 3 subtrees, H is D's child, E and G share parent C, and F, G, H, I, J have no children.

My answer

**Q3.** Major 172 / Activity 07-01

Consider the binary tree: 20: 10, 30; 10: 4, 15; 4: -, 9; 9: 7, -; 30: 25, 40; 40: 32, -; 32: -, 35. The proper descendants of node 30 are:

(A) 30, 25, 40, 32, 35

(B) 25, 40

(C) 40, 32, 35

(D) 25, 40, 32, 35

**Answer:** (D). Proper descendants exclude the node itself: all nodes below 30.

My answer

**Q4.** Major 172 / Activity 07-01

For the same tree as in the previous question (20: 10, 30; 10: 4, 15; 4: -, 9; 9: 7, -; 30: 25, 40; 40: 32, -; 32: -, 35), what are the levels of nodes 7, 15 and 20, respectively?

(A) 5, 3, 1

(B) 4, 2, 0

(C) 3, 1, 0

(D) 4, 2, 1

**Answer:** (B). Level = number of edges from the root: 20-10-4-9-7 has 4 edges, 20-10-15 has 2, the root is at level 0 (old key said 5, 3, 1, counting the root as level 1).

My answer

**Q5.** Major 172 / Activity 07-01

For the same tree (20: 10, 30; 10: 4, 15; 4: -, 9; 9: 7, -; 30: 25, 40; 40: 32, -; 32: -, 35), what are the heights of nodes 30, 25 and 32, respectively?

(A) 4, 1, 2

(B) 3, 1, 2

(C) 3, 0, 1

(D) 2, 0, 1

**Answer:** (C). Height = edges on the longest path down to a leaf: 30-40-32-35 is 3, 25 is a leaf (0), 32-35 is 1 (old key said 4, 1, 2, counting nodes instead of edges).

My answer

**Q6.** Major 172 / Activity 07-01

Is the same tree (20: 10, 30; 10: 4, 15; 4: -, 9; 9: 7, -; 30: 25, 40; 40: 32, -; 32: -, 35) full (decision) or complete?

(A) Full only.

(B) Complete only.

(C) Both full and complete.

(D) Neither full nor complete.

**Answer:** (D). Node 4 has one child (not full) and level 2 is not filled before level 3 (not complete).

My answer

**Q7.** Major 132 (Trees compilation)

Consider the binary tree: 15: 5, 16; 5: 3, 12; 12: 10, 13; 10: 6, -; 6: -, 7; 16: -, 20; 20: 18, 23. What are the degrees of nodes 20, 16, and 10, respectively?

(A) 2, 1, 2

(B) 2, 1, 1

(C) 3, 2, 2

(D) 2, 2, 1

**Answer:** (B). Degree = number of subtrees: 20 has two children, 16 and 10 have one each.

My answer

**Q8.** Midterm 242

Consider the binary tree: 32: 31, 26; 31: 24, 21; 24: 12, 15; 21: 13, -; 26: 19, 17. The node with key 31 is located at

(A) level 2 with height 2.

(B) level 1 with height 3.

(C) level 1 with height 2.

(D) level 2 with height 3.

**Answer:** (C). One edge from the root (level 1); longest path down 31-24-12 has 2 edges (height 2).

My answer

**Q9.** Midterm 222 (Trees compilation)

Choose the true statements from the following:

i. The minimum number of nodes of a binary tree with height $h$ is $h + 1$.
ii. The maximum number of nodes of a binary tree with height $h$ is $\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$.
iii. Tree traversal is the process of systematically visiting all the nodes in a tree and performing some processing at each node in the tree.
iv. Breadth-First Traversal and Depth-First Traversal are common methods to traverse a tree.

(A) None of the statements is true.

(B) Only statements iii and iv are true.

(C) Only statements i and ii are true.

(D) All the statements are true.

**Answer:** (D). A linear tree gives the minimum $h + 1$, a perfect tree the maximum $2^{h+1} - 1$; iii and iv are the slide definitions.

My answer

**Q10.** Midterm 222 (Trees compilation)

Choose the true statements from the following: i. Each perfect binary tree is a decision binary tree. ii. The number of leaves in a non-empty decision tree is one more than the number of nonterminal nodes. iii. The maximum height of a binary tree with $n$ nodes is $n - 1$.

(A) None of the statements is true.

(B) All the statements are true.

(C) Only statements i and ii are true.

(D) Only statements ii and iii are true.

**Answer:** (B). In a perfect tree every internal node has two children; ii is the slide theorem; a linear tree with $n$ nodes has height $n - 1$.

My answer

**Q11.** Midterm 242

Which of the following trees is a full binary tree, but is not a complete binary tree?

(A) 32: 31, 26; 31: 24, 21; 24: 12, 15; 21: 13, -; 26: 19, 17.

(B) 24: 18, 54; 18: 17, 21; 54: 36, 68; 68: 60, 70.

(C) 32: 31, 26; 31: 24, 21; 24: 12, 15; 21: -, 13; 26: 19, 17.

(D) None of the answers given is correct.

**Answer:** (B). Every node of (B) has 0 or 2 children, but its last level is not filled from the left. (A) is complete but not full; (C) is neither.

My answer

**Q12.** Midterm 242

The minimum number of nodes in a complete binary tree of height 10 is

(A) 1023

(B) 1024

(C) 2047

(D) 11

**Answer:** (B). Levels 0..9 are full ($2^{10} - 1 = 1023$ nodes) plus one node on level 10: $1024$.

My answer

**Q13.** Activity 07-02

Consider the binary tree: F: B, G; B: A, D; D: C, E; G: -, I; I: H, -. The breadth-first traversal of the tree is

(A) F B A D C E G I H

(B) A B C D E F G H I

(C) F B G A D I C E H

(D) F G B I D A H E C

**Answer:** (C). Level by level, left to right: F — B G — A D I — C E H.

My answer

**Q14.** Activity 07-02

For the same tree (F: B, G; B: A, D; D: C, E; G: -, I; I: H, -), the preorder traversal is

(A) F B A D C E G I H

(B) A C E D B H I G F

(C) F B G A D I C E H

(D) A B C D E F G H I

**Answer:** (A). N-L-R: F, then the subtree of B (B A D C E), then the subtree of G (G I H).

My answer

**Q15.** Activity 07-02

For the same tree (F: B, G; B: A, D; D: C, E; G: -, I; I: H, -), the postorder traversal is

(A) A B C D E F G H I

(B) A C E D B H I G F

(C) C E A D B H I G F

(D) H I G C E D A B F

**Answer:** (B). L-R-N: left subtree (A C E D B), right subtree (H I G), then F.

My answer

**Q16.** Quiz 3 (T232)

Consider the binary tree T: 20: 25, 30; 25: -, 35; 30: 40, 45; 40: -, 50; 45: 55, -. What is the in-order traversal of T?

(A) 35, 25, 50, 40, 55, 45, 30, 20

(B) 20, 25, 35, 30, 40, 50, 45, 55

(C) 25, 35, 20, 40, 50, 30, 55, 45

(D) 25, 35, 20, 50, 40, 30, 45, 55

**Answer:** (C). L-N-R: 25 35 — 20 — 40 50 30 55 45.

My answer

**Q17.** Quiz 3 V1 (T251)

List the sequence of nodes visited by postorder traversal of the tree: A: B, C; B: D, E; D: G, -; E: H, I; C: -, F; F: J, -.

(A) G D H I E B A J F C

(B) G D H I E B J F C A

(C) G D B H E I A C J F

(D) G D B H I E J F C A

**Answer:** (B). Left subtree G D H I E B, right subtree J F C, then the root A.

My answer

**Q18.** Midterm 202

Given the following instance methods of the BinarySearchTree class (`el` is the key field of BSTNode):

```
public void myTraversal(){
    myTraversal(root);
}
private void myTraversal(BSTNode node){
    if(node != null){
        System.out.print(node.el + " ");
        myTraversal(node.left);
        myTraversal(node.right);
        System.out.print(node.el + " ");
    }
}
```

What is the output of `tree.myTraversal();` if `tree` is the BST obtained by inserting 8, 3, 9, 7?

(A) 8 3 7 9 3 7 9 8

(B) 8 3 7 7 3 9 9 8

(C) 8 3 7 9 9 7 3 8

(D) 3 7 8 9 7 3 9 8

**Answer:** (B). Each node is printed on entering (preorder) and on leaving (postorder): 8 [3 [7 7] 3] [9 9] 8.

My answer

## Q19.

Midterm 242

Consider the general tree T: A: B, F, J; B: C, D, E; F: G, H; J: K, L, M, N; M: P, Q (children listed left to right). The depth-first pre-order traversal of T is

(A) C B D E A G F H K J L P M Q N

(B) A B C D E F G H J K L M N P Q

(C) A B F J C D E G H K L M N P Q

(D) A B C D E F G H J K L M P Q N

**Answer:** (D). Visit the node, then each child subtree left to right; P Q come right after M, before N.

My answer

## Q20.

Quiz 3 Ver A (T251)

Consider the general tree: 1: 2, 3, 4; 2: 5, 6; 4: 7. The post-order traversal of the tree is

(A) 1 2 3 4 5 6 7

(B) 1 2 5 6 3 4 7

(C) 5 2 6 1 3 4 7

(D) 5 6 2 3 7 4 1

**Answer:** (D). Children subtrees first, left to right, then the node: (5 6 2)(3)(7 4) 1.

My answer

### Q21.

Final 181 (Trees compilation)

The infix expression `((3-3) * (5+4) - (6 * (8-3) + (9/5)))` is represented as an expression tree. The prefix expression obtained by a preorder traversal of the tree is

(A) `- * - 3 3 + 5 4 + * 6 - 8 3 / 9 5`

(B) `- * - 3 3 + 5 4 + * 6 - 3 8 / 5 9`

(C) `* - 3 3 + 5 4 - + * 6 - 8 3 / 9 5`

(D) `- - * 3 3 + 5 4 + 6 * - 8 3 / 9 5`

**Answer:** (A). The root is the outer `-`; its left subtree is `*(-(3,3), +(5,4))` and its right subtree is `+(*(6,-(8,3)), /(9,5))`.

My answer

### Q22.

Final 181 (Trees compilation)

For the same expression `((3-3) * (5+4) - (6 * (8-3) + (9/5)))`, the postfix expression obtained by a postorder traversal of its expression tree is

(A) `3 3 - 5 4 + * 6 8 3 - * + 9 5 / -`

(B) `3 3 - 5 4 + 6 8 3 - * * 9 5 / + -`

(C) `3 3 - 5 4 + * 6 8 3 - * 9 5 / + -`

(D) `3 3 5 4 - + * 6 8 3 - * 9 5 / + -`

**Answer:** (C). L-R-N on the tree: `(3 3 -)(5 4 +) *`, then `(6 (8 3 -) *)(9 5 /) +`, then the root `-`.

My answer

### Q23.

Midterm 242

Consider the expression tree whose root is `/`. The root's left child is `+` with children 2 and `*`; the `*` node has left child `-` (children 10 and 1) and right child 2. The root's right child is `-` with children 8 and 3. What is the result of evaluating this expression tree?

(A) 4.4

(B) 8.75

(C) 20

(D) 4

**Answer:** (D). The tree computes $(2 + (10 - 1) \times 2) / (8 - 3) = 20/5 = 4$. Its inorder traversal `2 + 10 - 1 * 2 / 8 - 3` loses the parentheses.

My answer

### Q24.

Activity 07-02

Construct the expression tree for `a + (b * c) + d * (e + f)` (operators of equal precedence group from left to right). Which is the postorder traversal of the tree?

(A) `a b c * + d e f + * +`

(B) `a b c * d e f + * + +`

(C) `+ + a * b c * d + e f`

(D) `a b c * + d * e f + +`

**Answer:** (A). The root is the last +; its left subtree is `+(a, *(b,c))`, its right subtree is `*(d, +(e,f))`.

My answer

### Q25.

Activity 07-02

Given the binary tree 10: 11, 9; 11: 7, -; 9: 15, 8. Using the slides' binary-tree methods (delete: replace the deleted node by the last leaf in level order; insert: put the key at the first available position in level order), delete node 15, then insert 14. The resulting tree is:

(A) 14 is the left child of 9 and 8 is the right child of 9.

(B) 14 is the right child of 11; 9 has only a right child, 8.

(C) 14 is the right child of 9 and 8 is the left child of 9.

(D) 14 is the right child of 11; 9 has left child 8 and no right child.

**Answer:** (D). The last leaf 8 replaces 15 (so 8 becomes 9's left child); the first free spot in level order is then 11's right child.

My answer

### Q26.

Midterm 222 (Trees compilation)

One way to implement the getMin method of the BinarySearchTree class is as follows:

```
public T getMin(){
   if(isEmpty())
      throw new UnsupportedOperationException("Tree is empty.");
   else
      return getMin(root);
}
private T getMin(BTNode node){
   if(node.left == null)
```

```
      return (T) node.data;
   else
      return getMin(node.left);
}
```

(A) The minimum key is the key in the node that is the parent of the left-most node that has an empty left-subtree.

(B) The minimum key is the key in the node that is the parent of the right-most node that has an empty left-subtree.

(C) The minimum key is the key of the right-most node that has an empty left-subtree.

(D) The minimum key is the key of the left-most node that has an empty left-subtree.

**Answer:** (D). The method keeps going left until `node.left == null` and returns that node's key.

My answer

### Q27.

Quiz 3 Ver A (T242)

Which of the following statements is correct with respect to binary search trees?

(A) The node with the maximum key does not have a right child.

(B) The node with the maximum key does not have a left child.

(C) The node with the maximum key must be a leaf node.

(D) More than one answer given is correct.

**Answer:** (A). The maximum is the right-most node, so its right subtree is empty; it may still have a left child.

My answer

### Q28.

Major 172 (Trees compilation)

The BST below is built by inserting 20, 10, 30, 4, 15, 25, 40, 9, 32, 7, 35 into an empty tree. Where does the key 8 go when it is inserted?

(A) Left child of 9.

(B) Right child of 7.

(C) Right child of 4.

(D) Left child of 7.

**Answer:** (B). Path: 8¡20, 8¡10, 8¿4, 8¡9, 8¿7, and 7's right reference is null.

My answer

**Q29.** Activity 07-03

The BST is built by inserting 20, 15, 25, 10, 18, 16, 19. Where is the node with key 17 inserted?

(A) Right child of 16.

(B) Left child of 19.

(C) Right child of 15.

(D) Left child of 16.

**Answer:** (A). 17¡20, 17¿15, 17¡18, 17¿16, and 16's right reference is null.

My answer

**Q30.** Activity 07-03

Starting with an empty binary search tree, insert the keys 9, 5, 10, 6, 3, 4, 2 in the given order. The breadth-first traversal of the resulting tree is

(A) 9 5 10 6 3 4 2

(B) 9 5 10 3 6 4 2

(C) 9 5 10 3 6 2 4

(D) 9 3 10 2 5 4 6

**Answer:** (C). Tree: 9: 5, 10; 5: 3, 6; 3: 2, 4.

My answer

**Q31.** Major 131 (Trees compilation)

The keys 4, 7, 11, 14, 16, 18, 21, 23, 39 are inserted into an empty BST. Which insertion order gives the tallest BST (maximum height)?

(A) 16, 7, 23, 4, 11, 18, 39, 14, 21

(B) 23, 39, 11, 4, 21, 7, 16, 14, 18

(C) 4, 7, 11, 14, 16, 18, 21, 23, 39

(D) 7, 16, 39, 11, 4, 14, 23, 18, 21

**Answer:** (C). Sorted input gives a linear (degenerate) tree of height $n - 1 = 8$; the other orders give heights 3, 4 and 5.

My answer

**Q32.** Quiz 3 V1 (T251)

The binary search tree 5: 3, 9; 3: -, 4; 9: 7, 12; 7: 6, 8; 12: -, 20 was constructed by inserting a sequence of items into an empty tree. Which of the following input sequences will *not* produce this binary search tree?

(A) 5, 9, 3, 7, 6, 8, 4, 12, 20

(B) 5, 9, 3, 6, 7, 8, 4, 12, 20

(C) 5, 3, 4, 9, 12, 7, 8, 6, 20

(D) 5, 9, 7, 3, 8, 12, 6, 4, 20

**Answer:** (B). Inserting 6 before 7 makes 6 the left child of 9 instead of the left child of 7.

My answer

### Q33.

Major 172 (Trees compilation)

The BST is built by inserting 20, 10, 30, 4, 15, 25, 40, 9, 32, 7, 35. The root 20 is deleted by merging (the right subtree of 20 is attached as the right subtree of the maximum node of its left subtree, and the left child takes the place of 20). The result is:

(A) Root 10; 30 becomes the right child of 15.

(B) Root 15; 10 becomes the left child of 15.

(C) Root 25; 30 keeps 40 as its right child.

(D) Root 10; 30 becomes the right child of 9.

**Answer:** (A). The maximum of the left subtree (10, 4, 15, 9, 7) is 15, so 30's subtree hangs to the right of 15 and 10 becomes the root.

My answer

### Q34.

Quiz 3 (T232) / Quiz 3 V1 (T251)

Delete, by merging, the node 16 from the BST built by inserting 16, 11, 24, 15, 19, 13 (attach the right subtree to the maximum node of the left subtree). The preorder traversal of the resulting tree is

(A) 15, 11, 13, 24, 19

(B) 11, 15, 13, 24, 19

(C) 13, 11, 15, 24, 19

(D) 11, 13, 15, 24, 19

**Answer:** (B). 11 becomes the root; the maximum of the left subtree is 15, which receives 24 (with 19) as its right child: 11: -, 15; 15: 13, 24; 24: 19, -.

My answer

### Q35.

Quiz 3 Ver A (T242)

The BST is built by inserting 8, 5, 11, 2, 6, 10, 13, 4, 7, 9, 12, 14, 15. Key 11 is deleted by merging (attach the right subtree of 11 to the maximum node of its left subtree). After the deletion:

(A) 8's right child is 12, and 13 is the right child of 12.

(B) 8's right child is 10, and 13 becomes the right child of 9.

(C) 8's right child is 10, and 13 becomes the right child of 10.

(D) 8's right child is 9, and 10 becomes the right child of 9.

**Answer:** (C). The left subtree of 11 is 10 (with left child 9); its maximum is 10, so 13 (with 12, 14, 15) becomes 10's right child and 10 replaces 11.

My answer

### Q36.

Activity 07-03

The BST is built by inserting 13, 7, 19, 4, 10, 15, 22, 1, 6, 17, 20, 60. Delete 13 by copying, using the inorder *predecessor*. The result is:

(A) Root 15; 17 becomes the left child of 19.

(B) Root 10; 7 has no right child.

(C) Root 7; 19 becomes the right child of 10.

(D) Root 10; 6 becomes the right child of 7.

**Answer:** (B). The predecessor (maximum of the left subtree) is 10, a leaf; its key is copied into the root and the leaf is removed.

My answer

### Q37.

Midterm 222 (Trees compilation) / Activity 07-03

The BST is built by inserting 7, 2, 15, 1, 4, 8, 40, 3, 6, 9, 5. Delete the root 7 by copying, using the inorder *successor*. The result is:

(A) Root 6; 5 becomes the right child of 4.

(B) Root 2; 15 becomes the right child of 6.

(C) Root 8; 9 becomes the left child of 15.

(D) Root 9; 8 becomes the left child of 15.

**Answer:** (C). The successor (minimum of the right subtree) is 8, which has only a right child 9; 8 is copied into the root and 9 takes 8's old place.

My answer

### Q38.

Midterm 202

The BST is built by inserting 30, 24, 40, 16, 27, 35, 60, 18, 25, 29, 17, 26. Delete 24 by copying, using the inorder *predecessor*. The result is:

(A) 18 replaces 24, and 17 becomes the right child of 16.

(B) 18 replaces 24, and 17 becomes the left child of 16.

(C) 25 replaces 24, and 26 becomes the left child of 27.

(D) 16 replaces 24, and 18 becomes the left child of 27.

**Answer:** (A). The maximum of 24's left subtree is 18 (it has left child 17); 18 is copied up and 17 takes 18's place as 16's right child.

My answer

### Q39.

Major 132 (Trees compilation)

In the worst case, searching an imbalanced Binary Search Tree with $n$ nodes is

(A) $O(\log n)$

(B) $O(n \log n)$

(C) $O(n)$

(D) $O(2^n - 1)$

**Answer:** (C). A degenerate (linear) BST makes search walk through all $n$ nodes.

My answer

### Q40.

Quiz 3 V1 (T251)

Consider that $N$ distinct elements ($N > 3$) are inserted into an initially empty binary search tree (BST). Which of the following statements are true?

(A) The best case height of the resulting BST is $O(\log n)$

(B) The worst case height of the resulting BST is $O(n)$

(C) The worst case height of the resulting BST is $O(\log n)$

(D) More than one given answer is correct

**Answer:** (D). A balanced insertion order gives height $O(\log n)$; sorted input gives a linear tree of height $n - 1 = O(n)$. So both (A) and (B) hold.

My answer

## Part B: Programming

*Unless stated otherwise, use the slide classes:*

```
public class BTNode<T extends Comparable<T>> {
    protected T data;
    protected BTNode<T> left, right;
    public BTNode(T data) { this(data, null, null); }
    public BTNode(T data, BTNode<T> left, BTNode<T> right) {
        this.data = data; this.left = left; this.right = right;
    }
}
public class BinaryTree<T extends Comparable<T>> {
    protected BTNode<T> root;
    public boolean isEmpty() { return root == null; }
    // ...
}
public class BST<T extends Comparable<T>> extends BinaryTree<T> { /* ... */ }
```

### Q41.

Final 181 (Recursion compilation)

Write a recursive method `countLeaves` to count the number of leaves in a binary search tree. The method takes a node and returns the count as an integer. Add a public wrapper that counts the leaves of the whole tree.

**Answer:**

```
public int countLeaves() {
    return countLeaves(root);
}
private int countLeaves(BTNode<T> n) {
    if (n == null)
        return 0;
    if (n.left == null && n.right == null)
        return 1;
    return countLeaves(n.left) + countLeaves(n.right);
}
```

My answer

### Q42.

Quiz 3 Ver A (T251)

Develop a recursive method `size` such that when it takes as input the root of a binary tree, it returns the total number of nodes in that tree.

**Answer:**

```java
private int size(BTNode<T> node) {
    if (node == null) return 0;
    return 1 + size(node.left) + size(node.right);
}
```

My answer

## Q43. [new]

Write a `BinaryTree` instance method `reverseLevelOrderTraversal()` that prints the keys in reverse breadth-first order (the exact reverse of the level-order output; e.g. if the level order is K F U P M S T A R, print R A T S M P U F K). Use a `java.util.Queue` and a `java.util.Stack`.

**Answer:**

```java
public void reverseLevelOrderTraversal() {
    if (root == null) return;
    Queue<BTNode<T>> queue = new LinkedList<>();
    Stack<BTNode<T>> stack = new Stack<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        BTNode<T> node = queue.remove();
        stack.push(node);                    // instead of printing
        if (node.left != null) queue.add(node.left);
        if (node.right != null) queue.add(node.right);
    }
    while (!stack.isEmpty())
        System.out.print(stack.pop().data + " ");
    System.out.println();
}
```

My answer

## Q44. [new]

Write a recursive `BinaryTree` method `getHeight()` that returns the height of the tree, using the slide definition (height = number of edges on the longest path from the root to a leaf; the height of an empty tree is $-1$).

**Answer:**

```
public int getHeight() {
    return getHeight(root);
}
private int getHeight(BTNode<T> node) {
    if (node == null)
        return -1;
    return 1 + Math.max(getHeight(node.left), getHeight(node.right));
}
```

My answer

## Q45. [new]

Write two versions of the `BST` method `getMax()`: a recursive one (with a private helper) and an iterative one. Both return `null` for an empty tree.

**Answer:**

```
public T getMax() {                    // recursive
    if (isEmpty()) return null;
    return getMax(root);
```

```
}
private T getMax(BTNode<T> node) {
    if (node.right == null)
        return node.data;
    return getMax(node.right);
}

public T getMaxIterative() {           // iterative
    if (isEmpty()) return null;
    BTNode<T> current = root;
    while (current.right != null)
        current = current.right;
    return current.data;
}
```

My answer

## Q46. [new]

Write an *iterative* BST method `insert(T key)` that inserts a new key as a leaf. Throw an `IllegalArgumentExceptio` `key.")` if the key already exists.

**Answer:**

```
public void insert(T key) {
    BTNode<T> newNode = new BTNode<T>(key);
    if (root == null) { root = newNode; return; }
    BTNode<T> p = root, prev = null;
    while (p != null) {
        prev = p;
        int c = key.compareTo(p.data);
        if (c == 0)
            throw new IllegalArgumentException("Duplicate key.");
        else if (c < 0)
            p = p.left;
        else
            p = p.right;
    }
    if (key.compareTo(prev.data) < 0)
        prev.left = newNode;
    else
        prev.right = newNode;
}
```

My answer

## Q47. [new]

Write a `BinaryTree` method `isBST()` that returns `true` if the tree satisfies the BST ordering property (every key is greater than *all* keys in its left subtree and less than *all* keys in its right subtree). Note that comparing a node only with its two children is not enough.

**Answer:**

```
public boolean isBST() {
    return isBST(root, null, null);
}
// every key in this subtree must lie strictly between min and max (null = no bound)
private boolean isBST(BTNode<T> node, T min, T max) {
    if (node == null) return true;
    if (min != null && node.data.compareTo(min) <= 0) return false;
    if (max != null && node.data.compareTo(max) >= 0) return false;
    return isBST(node.left, min, node.data)
        && isBST(node.right, node.data, max);
}
```

My answer

## Q48. [new]

Write the `BST` method `treeSortDescending()` that prints all keys of the tree in decreasing order, using a reverse inorder traversal (R-N-L).

**Answer:**

```
public void treeSortDescending() {
    treeSortDescending(root);
    System.out.println();
}
private void treeSortDescending(BTNode<T> node) {
    if (node == null) return;
    treeSortDescending(node.right);
    System.out.print(node.data + " ");
    treeSortDescending(node.left);
}
```

My answer

## Q49. [new]

Write the `BinaryTree` (not BST) method `insert(T key)` that inserts the key at the first available position in level-order traversal, as in the slides.

**Answer:**

```
public void insert(T key) {
    BTNode<T> newNode = new BTNode<T>(key);
    if (root == null) { root = newNode; return; }
    Queue<BTNode<T>> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        BTNode<T> n = q.remove();
        if (n.left == null) { n.left = newNode; return; }
        q.add(n.left);
        if (n.right == null) { n.right = newNode; return; }
        q.add(n.right);
    }
}
```

My answer

## Q50. [new]

Write the `BST` method `deleteByMerging(T key)`. If the node has two subtrees, attach its right subtree as the right subtree of the maximum node of its left subtree, and let the left child take the deleted node's place (if the deleted node is the root, the left child becomes the root). Throw a `NoSuchElementException` if the key is not found.

**Answer:**

```
public void deleteByMerging(T key) {
    BTNode<T> p = root, parent = null;
    while (p != null && key.compareTo(p.data) != 0) {
        parent = p;
        if (key.compareTo(p.data) < 0) p = p.left;
        else p = p.right;
    }
    if (p == null) throw new NoSuchElementException("Key not found.");
    BTNode<T> replacement;
    if (p.right == null)            // leaf or only a left subtree
        replacement = p.left;
    else if (p.left == null)        // only a right subtree
        replacement = p.right;
    else {                          // two subtrees: merge
        BTNode<T> tmp = p.left;
        while (tmp.right != null)
            tmp = tmp.right;        // maximum of left subtree
        tmp.right = p.right;
        replacement = p.left;
    }
    if (parent == null) root = replacement;
    else if (parent.left == p) parent.left = replacement;
    else parent.right = replacement;
}
```

My answer